Implementation and Experimental Evaluation of a Geometric Quadrotor Controller for Hover and Circular Trajectory Tracking in ROS–Unity

Wenqian Zhang

Abstract— This report presents the complete setup, implementation, debugging, and experimental evaluation of a geometric quadrotor controller for MIT Visual Navigation for Autonomous Vehicles (VNAV) Lab 3. The software stack integrates ROS Noetic, a Unity/TESSE simulator, the TESSE ROS bridge, RViz, and a catkin workspace. In addition to the required quaternion, controllability, and aerodynamic-drag exercises, the work completes the supplied controller skeleton: ROS subscribers and publisher, desired- and current-state extraction, geometric position and attitude control on SO(3), and wrench-to-rotor allocation. The workspace compiled with all nine packages, after resolving a NumPy–SciPy incompatibility, and the full launch chain was verified experimentally. Two stationary-waypoint diagnostics and a circular-reference run were performed. For a 2.0 m commanded hover, 984 samples over 19.66 s yielded a mean altitude of 1.927 m and a standard deviation of 0.042 m. Rotor commands remained close to 50.219 rad/s on average. The final controller also produced sustained circular flight visible in RViz and Unity. Because the dynamic run was not recorded as synchronized reference/state data, that result is reported as qualitative evidence rather than an unsupported RMSE claim. The report documents unsuccessful tuning attempts, coordinate-frame and restart issues, limitations, and the exact submission artifacts required by the lab.

I. Introduction

A quadrotor is an underactuated rigid body: it has six configuration degrees of freedom but only four independent rotor inputs. Collective thrust directly controls acceleration along the body vertical axis, whereas horizontal motion requires the aircraft to tilt. Position, attitude, and actuator allocation therefore cannot be treated as unrelated problems. A useful tracking controller must convert a Cartesian reference into a physically consistent desired orientation, regulate that orientation, and allocate the resulting force and moment to four rotors.

MIT VNAV Lab 3 provides a simulation framework for studying this problem. A supplied trajectory publisher generates either a stationary reference or a hard-coded circle; a student controller receives that reference and simulated odometry, then publishes rotor angular speeds. The task is trajectory tracking rather than path planning: no map, obstacle search, or online trajectory optimizer is used. The control structure follows the geometric formulation of Lee et al. [2], which works directly with rotation matrices on SO(3) and avoids the singularities associated with Euler-angle feedback.

The work reported here contains four connected parts. First, the required individual technical exercises establish rotation conventions, rotor controllability, and body-frame drag compensation. Second, the complete ROS–Unity environment is built and verified. Third, the missing portions of the controller node are implemented, including communication, state extraction, geometric control, and rotor allocation. Fourth, the controller is evaluated through staged hover and circular-reference tests. The report deliberately separates measured evidence from interpretation. Build counts, sample statistics, and rotor-command ranges are quantitative. The circular flight is supported by screenshots and video, but no synchronized dynamic log was saved, so no numerical tracking-accuracy metric is invented.

The contributions of this laboratory work are:

- a reproducible ROS Noetic and Unity/TESSE setup procedure, including the dependency conflict and its repair;
- a complete implementation trace from the provided code skeleton to a running geometric controller;
- exact derivations for the individual deliverables and their link to the implemented software;
- experimental diagnostics for hover, command frequency, and motor-speed behavior;
- a documented tuning history, including a destabilizing gain trial, and a submission-completeness audit.

II. Requirements and System Architecture

A. Laboratory Deliverables

The handout divides the work into individual and team deliverables. The individual portion contains rotation-representation questions, analysis of normal and abnormal rotor layouts, and an aerodynamic-drag compensation derivation. The team portion requires the full source of controller_pkg under lab3/controller_pkg and a public video link stored in rviz_drone_flight.txt. The mathematical submission must be typeset as a PDF. This report integrates the individual derivations with the engineering record, but the source package and video-link file remain separate submission artifacts.

Wenqian Zhang, Student ID 24020036087. Laboratory report for MIT VNAV Lab 3 (ROS 1).

TABLE I
ROS Interfaces Used by the Controller

Direction	Topic	Message purpose
Subscribe	/desired_state	x_d, v_d, a_d, ψ_d
Subscribe	/current_state	x, v, q, ω
Publish	/rotor_speed_cmds	four rotor speeds

B. Closed-Loop Data Flow

Figure 1 summarizes the implemented loop. The supplied traj_publisher publishes a trajectory_msgs/MultiDOFJointTrajectoryPoint message on /desired_state. The controller subscribes to this topic and to nav_msgs/Odometry on /current_state. It publishes mav_msgs/Actuators on /rotor_speed_cmds. The TESSE bridge exchanges commands and state with Unity, while RViz visualizes both reference and measured motion.

The controller timer is configured for 100 Hz. That value is the attempted computation rate, not automatically the end-to-end feedback rate. During one hover diagnostic the measured state arrived at approximately 50 Hz, so repeated controller calls could operate on the most recently received odometry. This distinction is important when interpreting smoothness and apparent oscillation frequency.

III. Individual Technical Exercises

A. Quaternion and Rotation Representation

The first deliverable tests the relationship between rotation matrices, unit quaternions, and the skew-symmetric hat operator. The implementation uses Eigen quaternions in the coefficient order supplied by the ROS message constructor and then normalizes before conversion. If $q = [q_w, q_x, q_y, q_z]^T$ is a unit quaternion, its inverse is its conjugate,

$$q^{-1} = q^* = [q_w, -q_x, -q_y, -q_z]^T. \tag{1}$$

Quaternion multiplication is not commutative. For two rotations q_1 and q_2, applying q_1 and then q_2 corresponds to a convention-dependent ordered product; reversing the order generally yields a different attitude. This is why ROS message order, Eigen constructor order, and frame meaning must be checked together rather than copied mechanically.

The hat map converts a vector $a = [a_1, a_2, a_3]^T$ into

$$\widehat{a} = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}, \quad \widehat{a}b = a \times b. \tag{2}$$

Its inverse, the vee map, returns the vector encoded by a skew-symmetric matrix. For $R \in \mathrm{SO}(3)$, $R^T R = I$ and $\det R = 1$. Differentiating $R^T R = I$ gives $R^T \dot{R}$ skew-symmetric, motivating

$$\dot{R} = R\widehat{\omega}, \tag{3}$$

when ω is expressed in the body frame. The implemented controller uses precisely this convention after transforming the world-frame angular velocity from odometry into the body frame.

For attitude comparison, the scalar configuration error and vector error are

$$\Psi(R, R_d) = \frac{1}{2}\operatorname{tr}(I - R_d^T R), \tag{4}$$

$$e_R = \frac{1}{2}\left(R_d^T R - R^T R_d\right)^{\vee}. \tag{5}$$

These expressions are coordinate-free and remain well behaved away from the unavoidable topological ambiguity near a 180° relative rotation.

B. Normal Rotor Layout and Full Controllability

Let $u_i = \omega_i|\omega_i|$ denote the signed squared-speed variable. The sign-preserving definition matches the code path used to convert allocated inputs back to signed angular speeds. For the nominal layout, collective force and moments satisfy

$$\begin{bmatrix} f \\ M_x \\ M_y \\ M_z \end{bmatrix} = B_a \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix}, \tag{6}$$

with

$$B_a = \begin{bmatrix} c_f & c_f & c_f & c_f \\ lc_f & lc_f & -lc_f & -lc_f \\ -lc_f & lc_f & lc_f & -lc_f \\ c_d & -c_d & c_d & -c_d \end{bmatrix}. \tag{7}$$

For nonzero arm length l, thrust coefficient c_f, and drag coefficient c_d, elementary row operations yield four independent rows; equivalently, $\det B_a \neq 0$. Therefore $\operatorname{rank}(B_a) = 4$, and collective thrust plus three moments are independently controllable in the nominal configuration.

The signs in (7) are not universal constants. They depend on rotor numbering, arm geometry, spin direction, thrust direction, and the simulator's body axes. A controller can appear mathematically correct yet fail if a matrix from a paper is copied without reconciling these conventions. The final implementation therefore follows the lab-specific ordering.

C. Abnormal Rotor Layout

In the abnormal configuration specified by the exercise, the yaw row becomes

$$\begin{bmatrix} c_d & -c_d & -c_d & c_d \end{bmatrix}. \tag{8}$$

The roll and pitch rows remain as above, but the new yaw row is a scalar multiple of the pitch row:

$$\begin{bmatrix} c_d & -c_d & -c_d & c_d \end{bmatrix} = -\frac{c_d}{lc_f}\begin{bmatrix} -lc_f & lc_f & lc_f & -lc_f \end{bmatrix}. \tag{9}$$

Hence only three independent wrench directions remain and $\operatorname{rank}(B_a) = 3$. The vehicle loses independent yaw authority. This rank argument is stronger than a purely

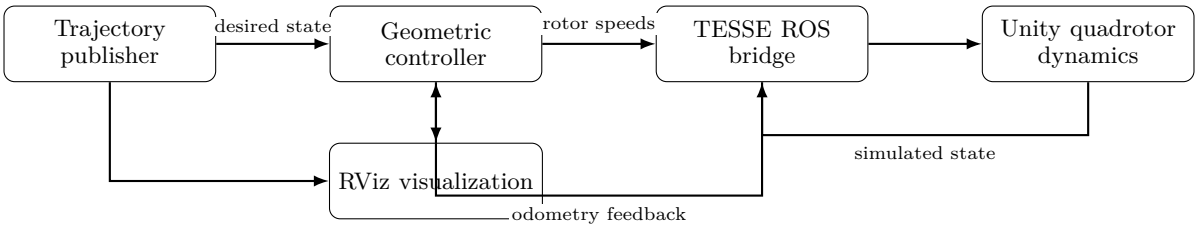


Fig. 1. Closed-loop architecture of the completed system. The controller is the only student-implemented feedback element; the simulator, bridge, message definitions, and reference generator are supplied components.

TABLE II
Experimental Software Environment

Component	Role
Ubuntu 20.04 VM	host environment
ROS Noetic	middleware and messages
catkin tools	workspace configuration/build
Unity/TESSE	quadrotor simulation
TESSE ROS bridge	simulator–ROS transport
RViz	frame and path visualization
Eigen	quaternion/matrix computation

visual explanation: it identifies exactly which command direction becomes linearly dependent.

D. Aerodynamic Drag Compensation

The exercise models drag in the body frame as

$$F^b_{\text{drag}} = \text{diag}(0.1, 0.1, 0.2) \begin{bmatrix} -v^b_x |v^b_x| \\ -v^b_y |v^b_y| \\ -v^b_z |v^b_z| \end{bmatrix}. \tag{10}$$

If measured velocity is available in the world frame, it is first rotated into the body frame,

$$v^b = R^T v^w. \tag{11}$$

The resulting drag is then rotated back into world coordinates. A feedforward compensation term modifies the desired total-force vector to

$$A_{\text{comp}} = -k_x e_x - k_v e_v + mge_3 + ma_d - RF^b_{\text{drag}}. \tag{12}$$

The sign in (12) cancels the predicted external drag force. This derivation is included as required theory; the final experimental controller uses the uncompensated force equation, so the flight results must not be described as validating drag compensation.

IV. Environment Preparation and Build

A. Software Platform

The experiment was conducted in an Ubuntu 20.04 virtual machine using ROS 1 Noetic. The main tools were catkin, Eigen, Python utilities required by the supplied interface, the Unity/TESSE simulator, the TESSE ROS bridge, and RViz. Keeping the simulator, bridge, ROS master, controller, and visualization in separate terminals made process state easier to diagnose.

B. Workspace Creation and Source Retrieval

The screenshot named “download code” records cloning the VNAV repository and selecting the 2023 ROS 1 branch. The terminal output confirms that the laboratory packages entered the local source tree before any controller modification was attempted.

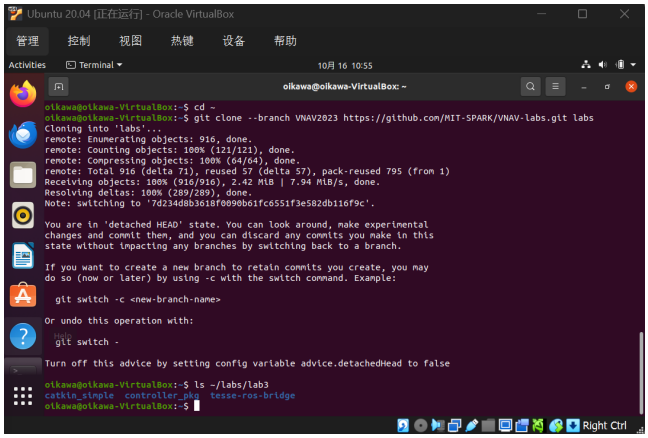


Fig. 2. Source retrieval: cloning the VNAV 2023 ROS 1 branch, matching the original note “download code.”

After the checkout, the catkin workspace was created and configured. The corresponding terminal lists the workspace paths and package layout, showing that controller_pkg, the bridge, message packages, and simulator interfaces were visible to the same build system.

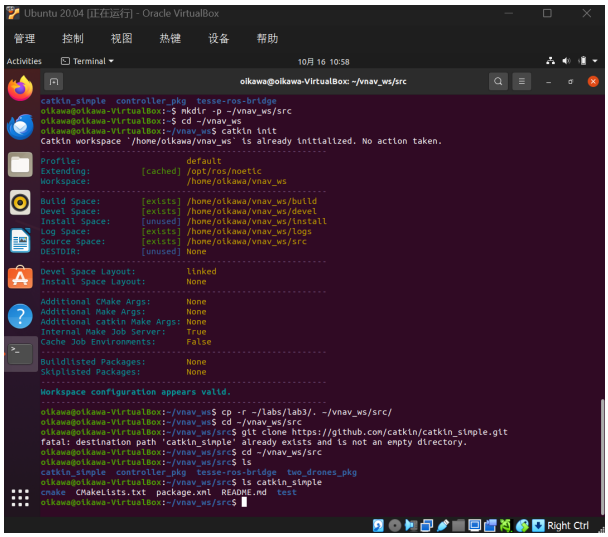


Fig. 3. ROS workspace creation and catkin configuration, matching the original note “create ROS workspace.”

The host then required the build and interface utilities shown in Fig. 4. Package installation output is included because missing compilers or Python tools would prevent the later catkin and bridge stages from being reproduced.

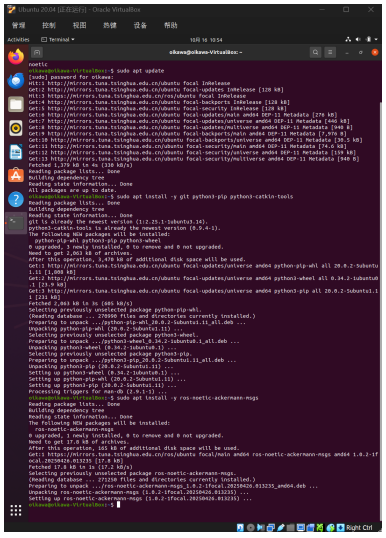


Fig. 4. Installation of the required build, ROS-interface, and Python tools.

The Lab 3 archive was downloaded and extracted separately from the ROS source tree. Figure 5 shows the simulator files being unpacked, which produced the Unity/TESSE executable used in the runtime experiment.

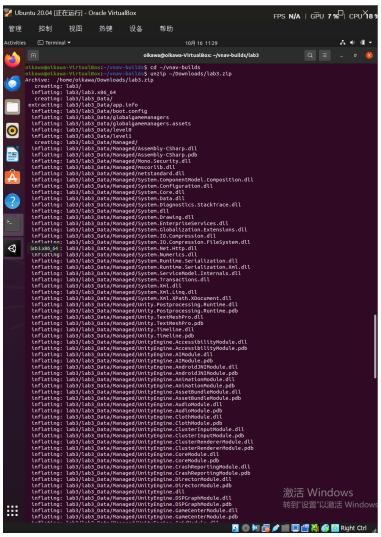


Fig. 5. Downloading and extracting the supplied Lab 3 Unity/TESSE simulator.

C. Dependency Conflict and Repair

The first complete setup did not run cleanly because the active Python environment contained NumPy 1.17.4 while SciPy 1.10.1 required a newer NumPy version. The terminal screenshot explicitly shows the compatibility warning and the subsequent installation of NumPy 1.24.4 in the active user environment.

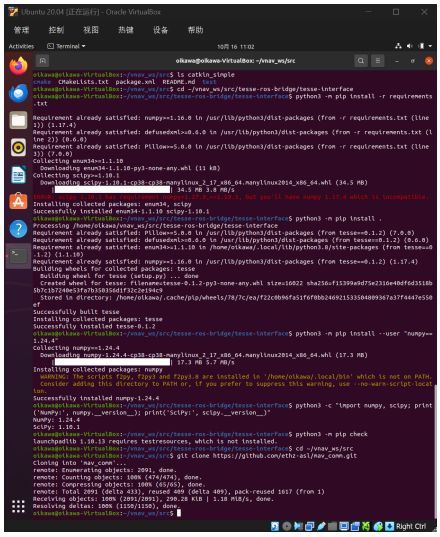


Fig. 6. Python dependency diagnosis and repair: the NumPy–SciPy incompatibility is identified and a compatible NumPy version is installed.

This episode also illustrated a common virtual-machine problem: package-manager state is layered. A system package, a user-level pip package, and a ROS-provided Python module can coexist. Therefore the corrective step was validated in the same terminal environment used for the bridge rather than assuming that an installation message proved the runtime import path.

D. Full Workspace Build

After dependency repair, catkin build compiled all nine workspace packages. The recorded full build took approximately 15.8 s and reported zero warnings, abandoned packages, or failures. A later rebuild after the controller and trajectory-mode changes completed in about 6.8 s. The shorter time is consistent with incremental compilation and should not be misinterpreted as a different source tree.

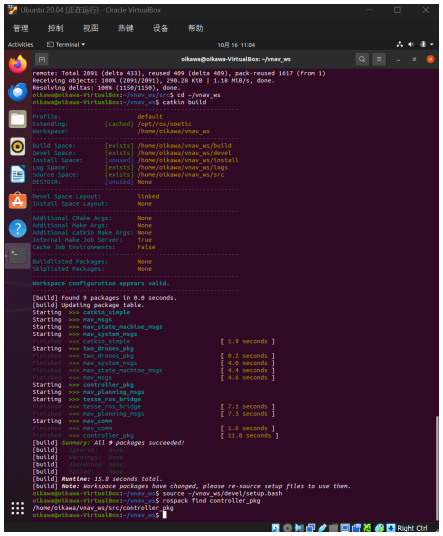


Fig. 7. Full catkin build summary showing 9 of 9 packages successful, with no failed or abandoned package.

V. ROS–Unity Runtime Integration

A. Launch Order

The runtime was started as an explicit sequence:

1) source the workspace environment and start roscore;
2) run the Unity/TESSE simulator executable;
3) start the TESSE ROS bridge;
4) launch the trajectory publisher and controller;
5) open RViz with the supplied Lab 3 configuration;
6) inspect topic frequency and message contents before interpreting motion.

This order separates failures cleanly. If no state topic exists, the bridge or simulator is investigated before controller mathematics. If state exists but no rotor command is published, controller state flags and callbacks are investigated. If both exist but Unity does not respond, the bridge mapping or simulator session is investigated.

B. Runtime Evidence

The screenshot noted as “run Lab 3” shows the Unity/TESSE scene with the quadrotor placed in the checkerboard laboratory environment. This confirms that the simulator executable was not merely extracted but actually launched.

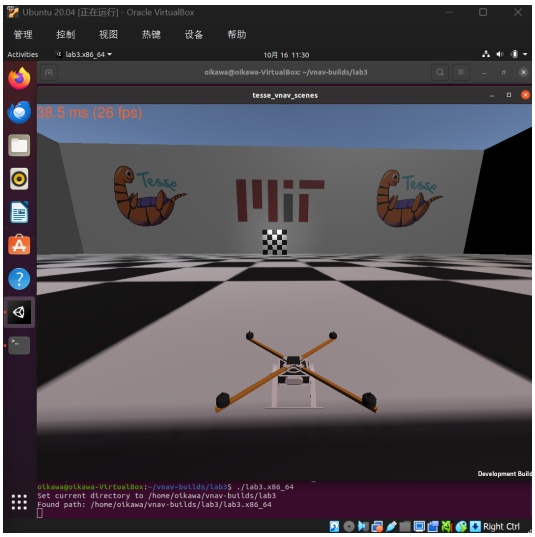


Fig. 8. Unity/TESSE running the Lab 3 scene with the simulated quadrotor visible.

The ROS master was started in a separate terminal. The roscore output in Fig. 9 provides the coordination service required before the bridge, trajectory publisher, and controller nodes can exchange messages.

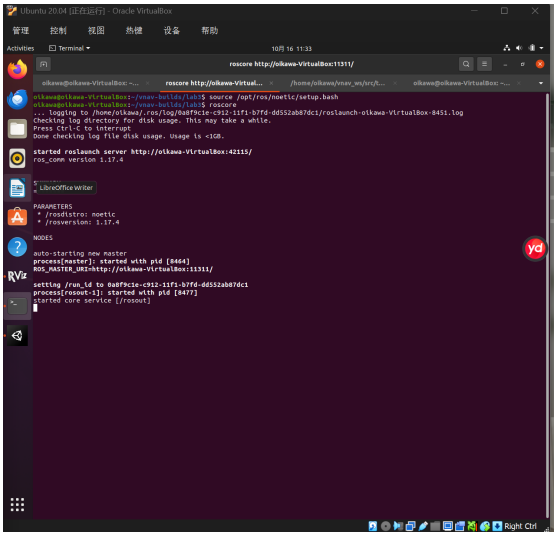


Fig. 9. ROS master startup in its dedicated terminal, matching the original note “start ROS Master.”

The Bridge terminal then reported its active configuration and continuous simulator-related output. This process is the transport layer that converts Unity/TESSE state into ROS messages and forwards rotor commands in the opposite direction.

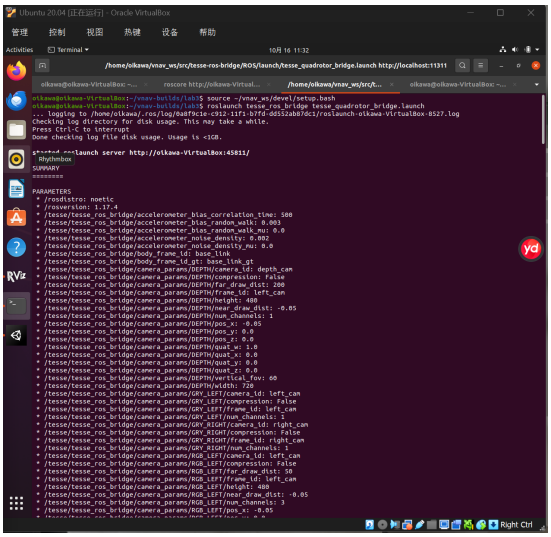


Fig. 10. TESSE ROS Bridge running between the simulator and ROS, matching the original “start ROS Bridge” note.

Finally, RViz was opened using the supplied Lab 3 configuration. The visible frame display and vertical marker in Fig. 11 confirm that the visualization was connected before the dynamic path was evaluated.

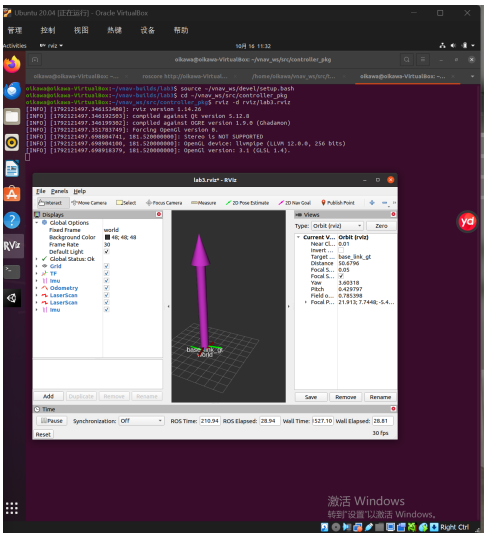


Fig. 11. RViz opened with the Lab 3 displays and coordinate-frame visualization.

C. Communication Verification

Before enabling dynamic tracking, three checks were performed. First, the desired-state topic was inspected to confirm that position, velocity, acceleration, and yaw-related transform fields were populated. Second, the current-state topic was monitored to confirm that Unity odometry changed with vehicle motion. Third, the rotor-command topic was checked after both callbacks had run. The controller uses two availability flags and returns from its timer callback until both a reference and an odometry message have been received. This prevents an uninitialized force or attitude command from reaching the simulator during startup.

The observed command update did not always equal the 100 Hz timer setting. One diagnostic indicated approximately 50 Hz state publication. This is not inherently a contradiction: ROS timer execution, bridge state rate, message inspection, virtual-machine load, and terminal visualization all affect observed rates. The practical conclusion was to distinguish controller attempt rate from fresh-state rate and to avoid claiming a 100 Hz closed-loop measurement without a synchronized log.

VI. Controller Software Implementation

A. From Skeleton to Complete Node

The supplied controller_node.cpp was divided into numbered tasks. In the original Part 1 screenshot, the class-level ROS callback handles and state-availability flags are still replaced by a FILL IN HERE marker.

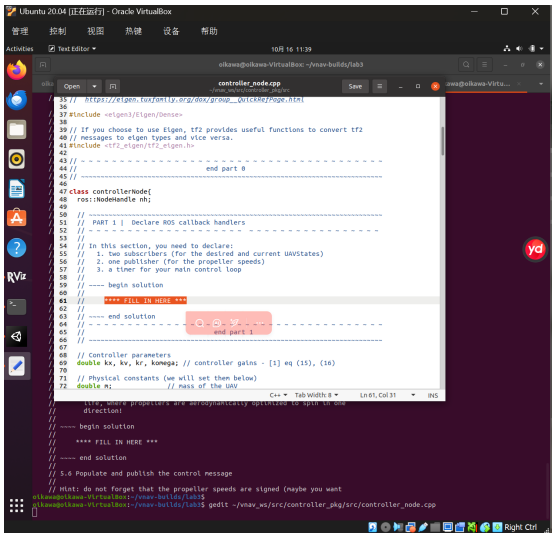


Fig. 12. Part 1 before modification: the controller class still contains the placeholder for ROS callback handles and state flags.

The completed Part 1 declares two subscribers, one actuator publisher, a control timer, and Boolean flags indicating whether desired and current states have arrived. These flags prevent control from running with uninitialized data.

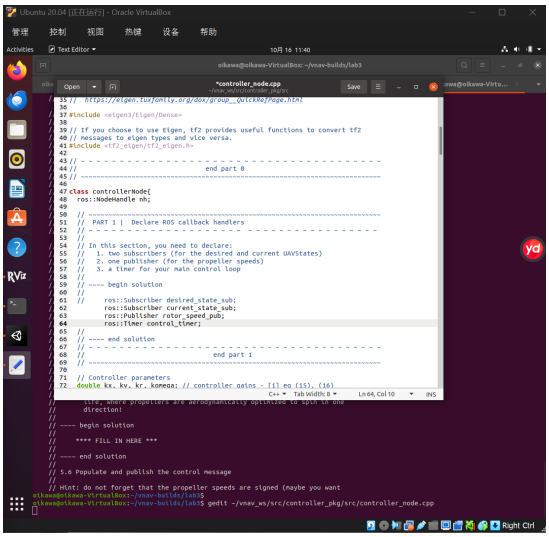


Fig. 13. Part 1 after modification: subscribers, publisher, timer, and the two message-availability flags are declared.

The original Part 2 screenshot shows the constructor task before implementation. Its comments explicitly request binding the desired-state and current-state callbacks and creating the timer at the frequency stored in hz.

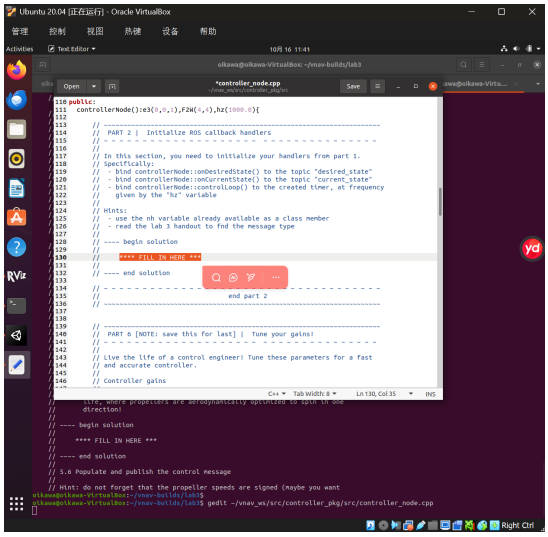


Fig. 14. Part 2 before modification: the constructor still contains the placeholder for callback binding and timer creation.

The completed Part 2 creates both subscribers, advertises /rotor_speed_cmds, and starts the periodic control callback. This converts the class declarations from Part 1 into active ROS communication objects.

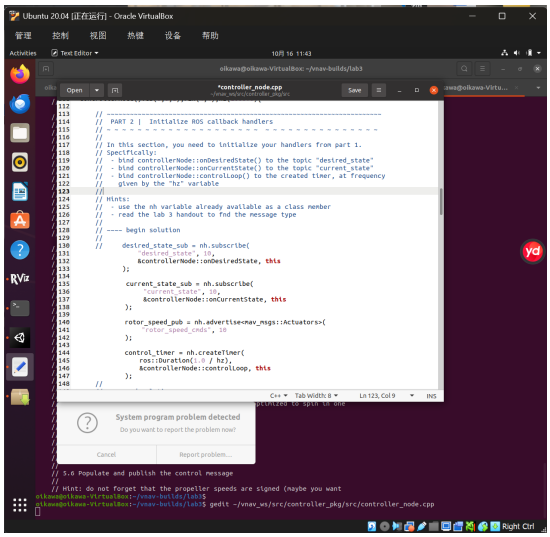


Fig. 15. Part 2 after modification: ROS subscribers, actuator publisher, and periodic control timer are initialized.

Part 3 reads the transform, velocity, and acceleration fields of the trajectory message. The screenshot records the actual extraction of desired position, desired velocity, desired acceleration, and yaw rather than only the final mathematical notation.

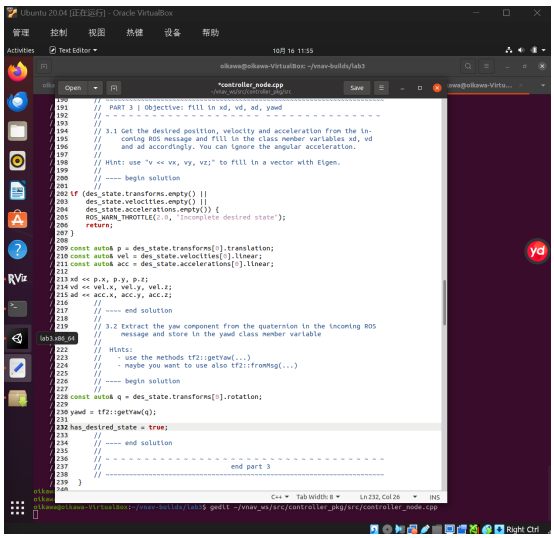


Fig. 16. Part 3 implementation: extraction of desired position, velocity, acceleration, and yaw from the trajectory message.

Part 4 processes the simulated odometry. The visible code normalizes the quaternion, converts it to R, and rotates the world-frame angular velocity into the body frame, directly addressing the coordinate convention used in the attitude law.

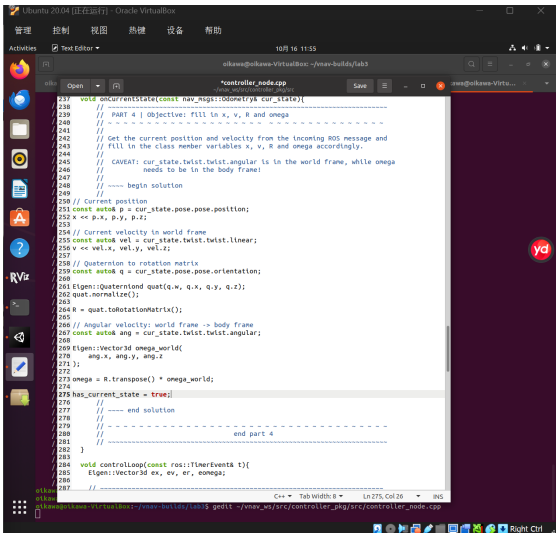


Fig. 17. Part 4 implementation: current-state extraction, quaternion normalization, rotation-matrix construction, and world-to-body angular-velocity conversion.

B. Desired-State Extraction

The trajectory point provides desired translation, velocity, acceleration, and a desired transform. The controller reads

$$x_d = [x_d, y_d, z_d]^T, \quad v_d = [\dot{x}_d, \dot{y}_d, \dot{z}_d]^T, \tag{13}$$

and

$$a_d = [\ddot{x}_d, \ddot{y}_d, \ddot{z}_d]^T. \tag{14}$$

The desired yaw ψ_d is recovered from the reference quaternion. Only yaw is prescribed independently; desired roll and pitch emerge from the commanded total-force direction. This is consistent with an underactuated vehicle whose body vertical axis must align with the required translational force.

C. Current-State Extraction and Frames

The odometry callback reads the position x, world-frame linear velocity v, and orientation quaternion q. The quaternion is normalized before conversion to the rotation matrix R. The simulator supplies angular velocity in the world frame, whereas the geometric moment equation expects a body-frame quantity. Therefore the implementation uses

$$\omega = R^T \omega_{\text{world}}. \tag{15}$$

This conversion fixed a subtle convention mismatch that can otherwise look like poor gain tuning. With mixed frames, the attitude-error vector and angular-velocity damping term do not describe the same physical coordinates.

D. Translational Control

Position and velocity errors are defined as

$$e_x = x - x_d, \qquad e_v = v - v_d. \tag{16}$$

The commanded world-frame force vector is

$$A = -k_x e_x - k_v e_v + mge_3 + ma_d. \tag{17}$$

The first two terms are proportional and derivative feedback. The third compensates gravity under the adopted axis convention, and the fourth is acceleration feedforward. This implementation does not include the optional drag term from (12).

E. Desired Attitude Construction

The desired body vertical direction is obtained from the normalized commanded force,

$$b_{3d} = \frac{A}{\|A\|}. \tag{18}$$

The desired heading vector in the horizontal plane is

$$b_{1c} = [\cos\psi_d, \sin\psi_d, 0]^T. \tag{19}$$

An orthonormal desired frame is constructed as

$$b_{2d} = \frac{b_{3d} \times b_{1c}}{\|b_{3d} \times b_{1c}\|}, \tag{20}$$

$$b_{1d} = b_{2d} \times b_{3d}, \tag{21}$$

$$R_d = [b_{1d}\ b_{2d}\ b_{3d}]. \tag{22}$$

This step is the geometric coupling between translation and attitude. Horizontal force components require b_{3d} to tilt, while ψ_d resolves the remaining rotation about that direction. In a production implementation, the normalization denominators should be guarded against near-zero values; the supplied circular and hover references do not drive the construction into that singular alignment.

F. Attitude and Moment Control

The attitude error is

$$e_R = \frac{1}{2}\left(R_d^T R - R^T R_d\right)^\vee. \tag{23}$$

The complete geometric law normally includes desired angular velocity and angular acceleration feedforward. In this laboratory implementation those desired rotational feedforward quantities are omitted, so

$$e_\omega = \omega. \tag{24}$$

The collective thrust and body moment are

$$f = A \cdot Re_3, \tag{25}$$

$$M = -k_R e_R - k_\omega e_\omega + \omega \times J\omega. \tag{26}$$

The gyroscopic term compensates the rigid-body coupling generated by $J\omega$. Because $J = I$ in the supplied controller constants, this cross product is zero numerically for the reported configuration; it is retained because it is part of the general dynamics and remains correct if a non-isotropic inertia is introduced later.

TABLE III
Model Constants and Final Controller Gains

Quantity	Symbol	Value
Mass	m	1.0 kg
Gravity	g	9.81 m/s^2
Arm parameter	l	0.30 m
Thrust coefficient	c_f	1.0×10^{-3}
Drag coefficient	c_d	1.0×10^{-5}
Inertia	J	$I_{3\times3}$
Position gain	k_x	1.0
Velocity gain	k_v	2.0
Attitude gain	k_R	2.0
Angular-rate gain	k_ω	1.0

G. Rotor-Speed Allocation

The commanded wrench $[f, M_x, M_y, M_z]^T$ is mapped to the signed squared-speed variables by solving

$$B_a u = \begin{bmatrix} f & M_x & M_y & M_z \end{bmatrix}^T. \tag{27}$$

The implementation uses a QR solve rather than an explicit matrix inverse. Rotor commands are recovered through

$$\omega_i = \operatorname{sign}(u_i)\sqrt{|u_i|}. \tag{28}$$

The sign-preserving square root avoids an invalid square root when transient allocation produces a negative signed input. However, the final code does not saturate rotor speeds. Thus feasibility is not guaranteed under large errors, and actuator clipping is an important future safety improvement.

VII. Experimental Procedure

A. Staged Validation Strategy

Testing proceeded incrementally rather than immediately commanding a circle. The stages were:

1) verify the ROS workspace and all package dependencies;
2) start Unity, ROS master, bridge, and RViz;
3) confirm reference, odometry, and actuator topics;
4) complete and rebuild each controller section;
5) switch the reference generator to a static target and test hover;
6) inspect altitude and rotor commands while adjusting gains cautiously;
7) restore the dynamic reference mode, rebuild, reset the simulator, and run the circular trajectory.

This sequence isolates faults. A stable hover is a prerequisite for interpreting circular tracking because the dynamic task contains the same vertical and attitude loops plus time-varying lateral commands.

B. Static-Reference Mode

The trajectory publisher contains a compile-time STATIC_POSE switch. It was temporarily set to 1 for hover diagnosis and later restored to 0 for the circular experiment. The original image file was noted as "change STATIC_POSE to 1," but the visible captured line is #define STATIC_POSE 0; the figure therefore documents the final restoration after the static test rather than claiming that the screenshot still shows 1. The static branch used a target altitude of 2.0 m and a yaw of approximately -2.902 rad.

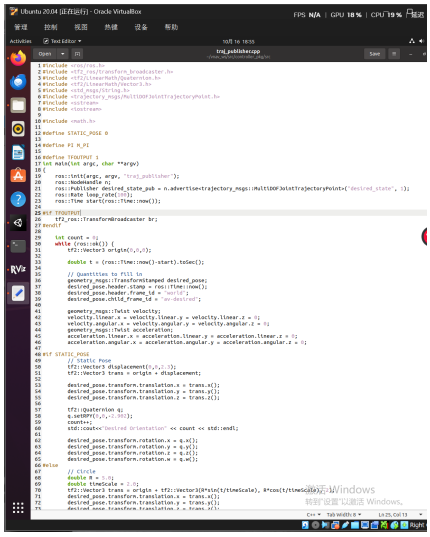


Fig. 18. Trajectory-mode edit. The visible final value is STATIC_POSE=0; the preceding hover stage used 1, consistent with the original file note.

Because this option is compiled into the trajectory publisher, changing it required rebuilding the affected packages. The terminal summary in Fig. 19 shows the successful incremental build before the dynamic experiment.

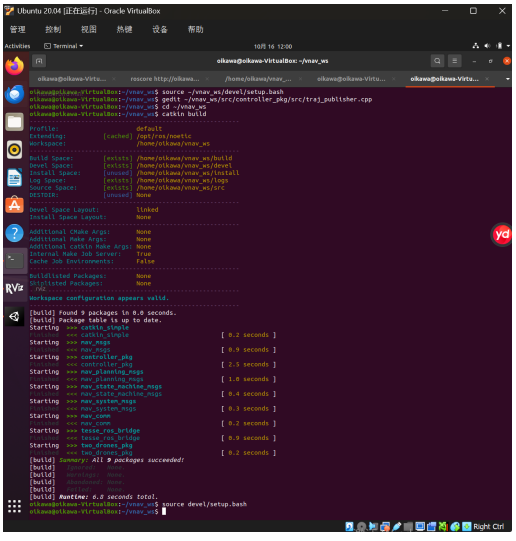


Fig. 19. Successful incremental rebuild after the controller and trajectory-mode changes.

The simulator state was reset between materially different trials. Restarting only the controller process does not reset Unity position, velocity, or attitude. Reusing a drifting simulator state had previously made a new gain set appear worse than it was. A clean comparison

TABLE IV
Observed Gain-Tuning Outcomes

Trial	Change	Observation and decision
Baseline	$k_x = 1, k_v = 2$	Usable hover and dynamic motion; retained.
Higher k_x	$k_x = 2$	Visible tilt and x drift to about -1.286 m; rejected.
Height test	$z_d = 2.0$ m	Longer diagnostic interval; small mean bias remained.
Final	static to circle	Reset simulator, restore STATIC_POSE=0, rebuild.

therefore requires the same initial simulator state, the same reference, and the same logging interval.

C. Gain-Tuning Procedure

The screenshot named "Part 6" shows the controller parameter area and the declarations of k_x, k_v, k_R, and k_ω near the tuning instructions. It documents where the gain-tuning stage connects to the controller class, even though the numerical values are supplied by the launch configuration.

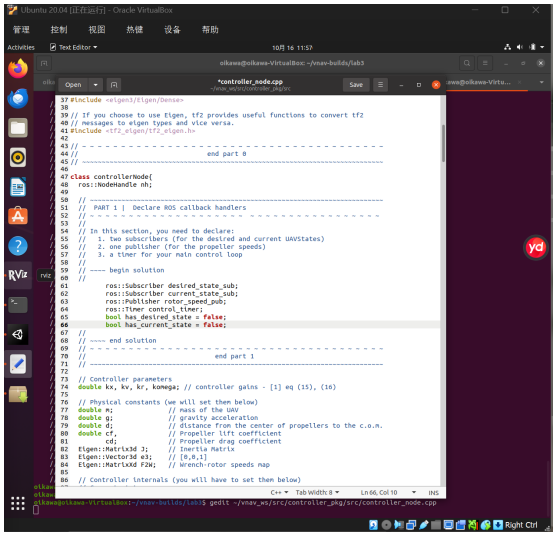


Fig. 20. Part 6 tuning context: controller gain declarations and the surrounding parameter block in controller_node.cpp.

The final launch parameters were $k_x = 1$, $k_v = 2$, $k_R = 2$, and $k_\omega = 1$. Tuning observations were qualitative rather than a controlled grid search. In particular, increasing k_x to 2 produced a visible tilt and lateral drift, with the x position reaching roughly -1.286 m in one trial. This result demonstrated that a larger proportional gain is not automatically an improvement. Stronger position feedback demands a larger attitude change; if attitude response, frame conventions, simulator initialization, or damping do not match, lateral motion can grow.

D. Circular Reference

In dynamic mode, the supplied publisher generates a circle with radius $r = 5$ m, altitude $z_d = 2$ m, and time scale 2. The implemented source is equivalent to

$$x_d(t) = 5\cos(t/2), \qquad y_d(t) = 5\sin(t/2), \tag{29}$$

$$z_d(t) = 2, \qquad \psi_d(t) = -t/2. \tag{30}$$

The reference period is therefore approximately $4\pi = 12.57$ s and the tangential speed is $r/2 = 2.5$ m/s. Analytical velocity and acceleration are published with the position, providing feedforward inputs to (17). This is a substantially harder test than hover because the controller must continuously tilt, generate centripetal acceleration, and coordinate changing yaw with translation.

VIII. Experimental Results

A. Build and Communication Results

The final workspace build succeeded for all nine packages. The Unity simulator, ROS master, bridge, controller, trajectory publisher, and RViz operated simultaneously. Desired-state, current-state, and rotor-command messages were present. These results establish software integration but do not by themselves establish tracking quality; the hover and circular observations provide the control-level evidence.

B. Initial Hover Diagnostic

The first stationary test recorded 596 state samples over approximately 11.9 s. Mean altitude was 1.7253 m, with a minimum of 1.6796 m and maximum of 1.7848 m. Relative to a 2.0 m reference, the mean error was -0.2747 m. Observed vertical velocity ranged from approximately -0.1710 to 0.0874 m/s. The bounded altitude band indicated closed-loop regulation, but the negative mean error showed that the vehicle did not converge to the requested height.

The test was useful even though it was not the final experiment. It separated catastrophic instability from a model or feedback bias: the vehicle remained in a limited altitude region rather than diverging, yet the equilibrium was systematically low. Possible explanations include thrust-scale mismatch, mass or gravity convention differences, vertical drag, bridge timing, or omitted integral action.

C. Extended 2.0 m Hover Diagnostic

A second, longer diagnostic retained the $z_d = 2.0$ m target. It collected 984 samples over 19.66 s, corresponding to roughly 50 state samples per second. Table V summarizes the measurements. The mean altitude was 1.927 m and standard deviation was 0.042 m. The range, 1.843–2.007 m, again indicated bounded motion. The mean vertical error was approximately -0.073 m. Maximum observed ascent and descent rates were 0.119 and -0.223 m/s, respectively.

No reliable oscillation period is reported. A visual waveform can contain startup transient, simulator reset effects, message-rate aliasing, and genuine closed-loop oscillation simultaneously. Estimating a period requires

TABLE V
Stationary-Waypoint Measurements

Metric	Test A	Test B
Commanded altitude	2.0 m	2.0 m
Samples	596	984
Duration	11.9 s	19.66 s
Mean altitude	1.7253 m	1.927 m
Minimum altitude	1.6796 m	1.843 m
Maximum altitude	1.7848 m	2.007 m
Altitude standard deviation	not saved	0.042 m
Mean vertical error	−0.2747 m	−0.073 m
Max. ascent velocity	0.0874 m/s	0.119 m/s
Max. descent velocity	−0.1710 m/s	−0.223 m/s

a timestamped signal segment after transients and a defined method such as peak detection or spectral analysis. That data was not preserved, so the report avoids a false-precision claim.

D. Rotor-Command Behavior

One actuator record contained 416 commands. The mean rotor angular speed was approximately 50.219 rad/s, and the observed command range was 49.770–50.611 rad/s. The narrow range is consistent with the vehicle operating near a hover equilibrium and with the simple symmetric model. It does not prove actuator feasibility in aggressive flight because no explicit saturation is implemented and the dynamic-run commands were not saved for the same analysis.

The ideal collective hover estimate provides a useful scale check. With four equal rotors and the implemented c_f, one expects

$$\omega_{\text{hover}} \approx \sqrt{\frac{mg}{4c_f}}, \tag{31}$$

subject to the simulator's exact coefficient convention. The recorded command magnitude is of the expected order, supporting the conclusion that the allocation and units are internally plausible even though a vertical steady-state bias remains.

E. Circular Tracking Result

After restoring dynamic mode, rebuilding, and resetting the simulator, the vehicle produced sustained curved motion and a visible circular path in RViz while Unity propagated the quadrotor. Figure 21 is the final experimental screenshot. The result demonstrates that reference publication, state feedback, geometric force-direction construction, moment control, allocation, and simulator command transport all operated together for a time-varying trajectory.

The appropriate conclusion is "successful qualitative circular tracking," not "accurate tracking within a specified tolerance." A still image cannot determine phase lag, cross-track RMSE, maximum position error, or lap-to-lap repeatability. The linked video strengthens the evidence of continuous motion, but quantitative performance would require logging $x_d(t)$ and $x(t)$ on a common clock.

IX. Engineering Challenges and Solutions

A. Environment and Toolchain Problems

The NumPy–SciPy incompatibility was repaired by installing a compatible NumPy version in the environment actually used by the bridge. A separate workflow problem was duplicate cloning or placing code in an unexpected directory. The practical solution was to inspect the workspace source tree before repeating a clone and to verify package discovery through catkin. Repeating commands without checking filesystem state can create nested repositories and make later paths misleading.

File transfer between host and virtual machine also complicated the workflow. Screenshots, source files, and logs were produced in different environments. Consistent filenames and a dedicated evidence directory reduced the risk of citing an obsolete image. For the final report, filenames were converted to stable ASCII names so that LaTeX could include them reliably on Windows and Linux.

B. ROS and Unity Communication

At one point ROS topics stopped publishing normally. The investigation treated the system as a chain: ROS master, Unity process, bridge connection, controller callbacks, and topic rates. Restarting only the controller was insufficient when the bridge or simulator session was stale. The recovery was to terminate the affected processes, restart the launch chain in order, and verify each topic before enabling control.

A second issue was that restarting the controller did not reset Unity. This distinction matters experimentally. The controller has internal message flags and gains; Unity holds the physical state. A new controller process can therefore inherit a vehicle that is already displaced, tilted, or moving. Resetting the simulator was added to the protocol for comparisons between gain sets and between static and dynamic modes.

C. Coordinate and Allocation Conventions

The angular-velocity frame mismatch was corrected by applying R^T to the world-frame vector. The rotor allocation matrix was built from the laboratory's motor ordering and spin signs rather than copied directly from the reference paper. These issues demonstrate why geometric formulas are only part of a working controller: interface conventions determine whether the mathematics corresponds to the simulated body.

The yaw reference also differed noticeably from the simulator's initial attitude. Large initial yaw error can command strong moments and obscure vertical or lateral tuning. Static testing made that behavior visible before the circle added continuous yaw change. A more systematic future protocol would initialize the reference yaw to the measured yaw, stabilize translation, and only then introduce the desired yaw trajectory.

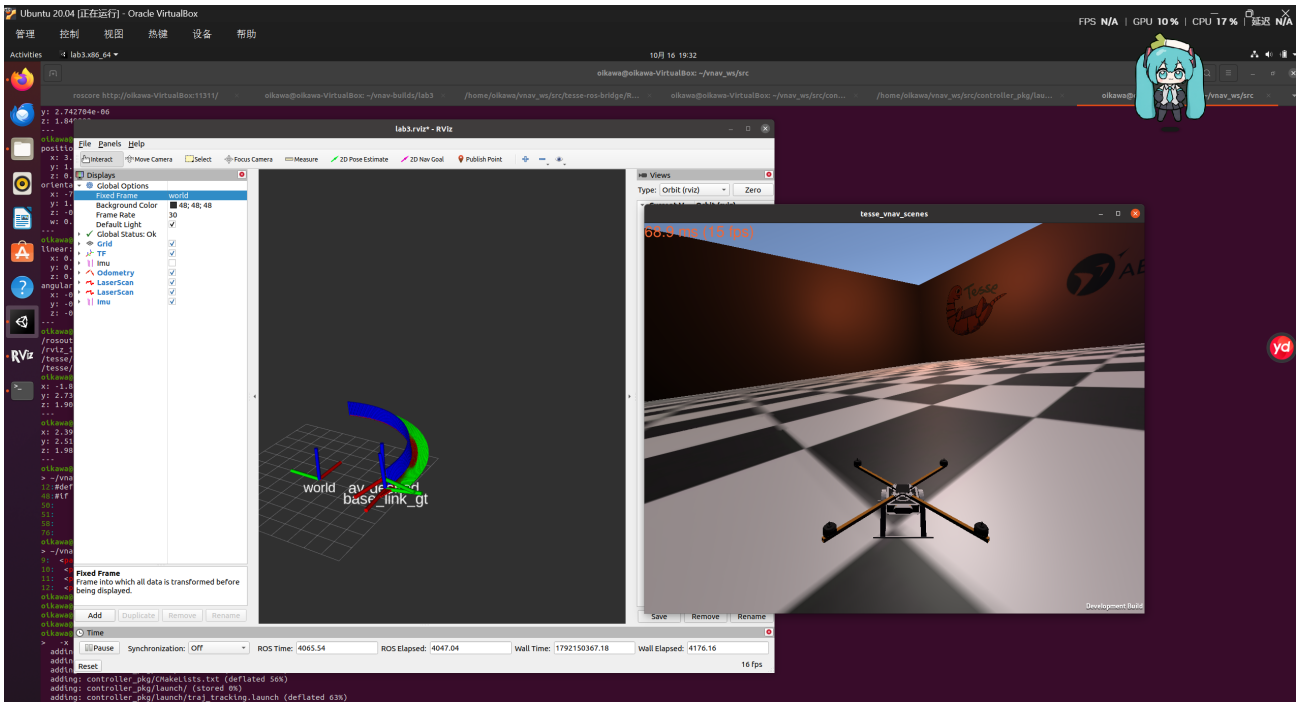


Fig. 21. Final circular-reference experiment. RViz shows the desired/current frames and a sustained curved trail, while Unity shows the simulated vehicle. This image is qualitative trajectory evidence; synchronized desired and measured states were not recorded for a numerical RMSE calculation.

D. Tuning and Model Mismatch

The higher-k_x trial showed that aggressive gains can excite coupling rather than reduce error. The persistent altitude bias in both hover tests suggests that feedback gains alone are not the only issue. A proportional–derivative position loop reaches a biased equilibrium when the modeled feedforward force differs from the simulator's effective force. Integral action, calibrated thrust coefficients, or disturbance estimation could remove such a bias, but each should be introduced only after confirming coordinate, timing, and reset consistency.

X. Discussion

A. What the Experiments Establish

The results establish that the implemented controller closes the loop through the complete ROS–Unity stack and can produce bounded hover and sustained circular motion. They also show that the nominal allocation is consistent enough with the simulator for all four rotors to operate near an expected hover magnitude. The experiments do not establish optimality, hardware transfer, drag-compensation benefit, or a specific circular RMSE.

Static and dynamic tracking expose different aspects of the design. Hover primarily tests gravity compensation, vertical feedback, equilibrium thrust, and attitude stabilization. Circular tracking additionally tests feedforward acceleration, continuous desired-attitude reconstruction, lateral position control, and changing yaw. Passing hover is therefore necessary but not sufficient for strong dynamic tracking.

B. Interpretation of the Altitude Bias

Both static tests used a 2.0 m target and exhibited a negative mean altitude error, although the longer test reduced the magnitude from -0.2747 m to about -0.073 m. This pattern is compatible with a systematic force mismatch, different transient coverage, or both. If the true mapping from rotor speed to thrust differs from c_f in the controller, the gravity term mge_3 produces the wrong equilibrium command. A PD controller can balance the residual through a nonzero position error, exactly the observed behavior. Other possibilities include simulator drag, timing delay, a mismatch in mass or frame convention, and an initial transient included in the mean.

The rotor-command range was narrow, suggesting the controller was not rapidly alternating between extreme commands in the recorded hover interval. However, without simultaneously logged force, attitude, and state, the data cannot uniquely identify the cause. A good next experiment would record all four rotor commands, z_d, z, $\dot{z}$, and orientation with common timestamps, discard a defined transient window, and compare the measured equilibrium command against the model prediction.

C. Limitations

The main experimental limitation is missing synchronized data for the circular run. The final screen capture verifies a path and continuous operation but

TABLE VI
Problem–Diagnosis–Resolution Traceability

Observed problem	Diagnosis	Action and lesson
Python compatibility warning	NumPy 1.17.4 incompatible with SciPy 1.10.1	Install NumPy 1.24.4 in the active runtime and retest the interface.
Possible duplicate source tree	Clone repeated without checking existing path	Inspect workspace layout and package discovery before cloning again.
Topics stop updating	Stale master/bridge/simulator process or broken launch order	Restart the affected chain and validate topics one stage at a time.
New gain test starts from drifted state	Controller restart does not reset Unity dynamics	Reset simulator state for comparable trials.
Unexpected attitude response	Angular velocity and attitude error expressed in different frames	Transform ω_{world} with R^T before applying body-frame feedback.
Incorrect wrench response risk	Motor ordering/signs differ from generic paper diagram	Derive and use the lab-specific allocation matrix.
Lateral divergence with $k_x = 2$	Position loop requests stronger tilt than the full loop tolerates	Reject the isolated gain increase and return to the stable baseline.
Persistent low hover altitude	Likely model/feedforward bias without integral correction	Report the bias; propose calibration, integral control, and drag compensation as future work.

cannot quantify reference error. The gain study was observational rather than systematic, and each trial did not share a formally scripted initialization. The code lacks actuator saturation, integrator anti-windup, singularity guards for desired-axis normalization, and stale-message timeouts. Desired angular velocity and angular acceleration feedforward are omitted. The aerodynamic drag compensation is derived but not integrated into the tested controller.

The virtual-machine environment adds nondeterministic scheduling and bridge latency. The 100 Hz control timer and approximately 50 Hz observed state rate mean some control calculations may reuse a state sample. This does not invalidate the experiment, but it limits claims about bandwidth and makes real-time timing measurements important for future evaluation.

XI. Future Work

A. Quantitative Trajectory Evaluation

The highest-priority improvement is a synchronized ROS bag containing desired state, odometry, rotor commands, and timestamps. For N samples, position RMSE should be computed as

$$\mathrm{RMSE}_p = \sqrt{\frac{1}{N}\sum_{k=1}^{N} \|x_k - x_{d,k}\|^2}, \tag{32}$$

with separate vertical and horizontal errors, maximum error, phase lag, and lap-to-lap repeatability. The first transient lap should be reported separately from steady periodic operation.

B. Systematic Tuning and Robustness

Gain tuning should vary one factor at a time from a repeatable initial state and save a metric table for every run. Motor saturation and command-rate limits should be added before testing more aggressive gains. A stale-state timeout should suppress commands when the bridge stops updating. Desired rotational feedforward should be implemented for rapidly changing trajectories, and normalization denominators should be protected.

C. Model and Drag Compensation

The effective thrust coefficient can be calibrated from hover data instead of assumed. After this baseline calibration, the body-frame drag model in (10) can be added to (12). Comparing identical circular runs with and without compensation would then determine whether cross-track error and phase lag improve. Integral action or a disturbance observer could address steady bias, but saturation-aware design is needed to prevent windup.

D. Beyond a Fixed Circle

The laboratory publisher supplies a fixed analytical trajectory. A future system could accept time-parameterized paths from a planner, enforce dynamic feasibility, and test obstacle-rich environments. Such an extension would add planning and perception on top of the tracking controller; it should not be confused with what the present experiment implements.

XII. Submission Coverage and Reproducibility

Table VII maps each requirement or evidence category to an artifact. The mathematical report, controller source, video-link text, and repository placement must all be checked separately before submission. The handout's published deadline belongs to the 2023 course offering and should not be assumed to be the current local deadline.

For reproduction, the essential sequence is: prepare the ROS Noetic workspace; place the VNAV packages in its source tree; install compatible Python dependencies; build and source the workspace; start ROS master, Unity, and the bridge; launch the controller/reference pair; open RViz; verify all three controller topics; test static mode from a reset simulator state; restore STATIC_POSE=0,

TABLE VII
Requirement and Evidence Coverage

Item	Artifact and final check
Derivations	Report PDF, Sec. III; verify personal-repository placement.
Controller source	lab3/controller_pkg; launch and RViz files included.
Flight video	rviz_drone_flight.txt; verify anonymous public access.
Team repository	Confirm the actual TEAM_N name and remote repository.
Build evidence	Figs. 7, 19; 9/9 packages successful.
Runtime evidence	Figs. 8–11; simulator, master, bridge, and RViz shown.
Code evidence	Figs. 12–20; before/after ROS I/O, parsing, and tuning context shown.
Hover data	Table V; two tests and limitations included.
Circular flight	Fig. 21 plus video; qualitative evidence only.
Deadline/rubric	Check the current portal; the handout date is historical.

rebuild, reset, and execute the circle. This sequence turns the screenshots into a repeatable procedure instead of a collection of isolated terminal images.

XIII. Conclusion

This laboratory completed substantially more than the two mathematical proofs. It established a ROS Noetic and Unity/TESSE environment, repaired a real dependency conflict, built nine packages, verified the complete runtime process chain, implemented every major section of the geometric controller, and executed both hover and circular-reference tests. The individual exercises explained quaternion and rotation conventions, proved full nominal control allocation and rank loss in the abnormal layout, and derived a body-frame aerodynamic-drag compensation term.

Experimentally, the controller maintained bounded stationary flight and produced sustained circular motion. The longer 2.0 m hover test yielded a mean of 1.927 m with 0.042 m standard deviation, revealing a residual mean vertical bias of about -0.073 m. A gain increase to $k_x = 2$ worsened lateral behavior and was rejected, illustrating the coupled nature of quadrotor tuning. The final circle is supported by RViz/Unity evidence and video, but not by synchronized error logs; the report therefore avoids overstating precision. The resulting package provides a complete technical narrative, the required source and link artifacts, and a clear plan for quantitative, saturation-aware, and drag-compensated future evaluation.

References

[1] MIT VNAV Staff, "Lab 3: Quadrotor Control," Visual Navigation for Autonomous Vehicles, Handouts 2023 (ROS 1), 2023.

[2] T. Lee, M. Leok, and N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE(3)," in Proc. 49th IEEE Conf. Decision and Control, 2010, pp. 5420–5425, doi: 10.1109/CDC.2010.5717652.

[3] Open Robotics, "ROS Noetic Ninjemys documentation," ROS 1 documentation.

[4] G. Guennebaud, B. Jacob, et al., "Eigen v3," linear algebra library documentation.